Mistral

Etiqueta de modelo: (no observada)
Leida: 2026-11-09T21:23:30.965Z
Caracteres: 17404
Fuentes citadas (verificables): 54
Enlaces en el DOM: 54
Fin de respuesta: inferido

Pregunta

Contexto: ChatCouncil es una app de investigación que abre diez páginas web reales de proveedores de IA a la vez, cada una en su propio webview nativo con la sesión del usuario; escribe en sus campos de texto, lee sus respuestas del DOM, adjunta archivos y arma un informe. Se está reescribiendo con el núcleo en Rust y un solo código para Windows, macOS, Linux, Android e iOS (escritorio, móvil y tableta), con el rendimiento (RAM, CPU, tiempo de arranque y tamaño) como prioridad absoluta.

Ya medido en Windows, con diez webviews nativos de WebView2 en la misma máquina: Flutter con plugins que dibujan por textura gasta unos 5 puntos de CPU extra cuando las páginas redibujan; Flutter con ventanas nativas y Slint con ventanas nativas (wry) gastan casi cero en el proceso propio; Slint arranca en 0,76 s contra 3,35 s de Flutter, pesa 11 MiB contra 25,5 MiB y usa 74 MB de RAM en el proceso propio contra 122 MB. Se descartaron Dioxus (su interfaz sería un webview más) y Rune (no tiene Android ni iOS).

Pregunta: ¿qué opciones de interfaz y de manejo de varios webviews nativos simultáneos, viables hoy para esas cinco plataformas con un solo código y un núcleo en Rust, no se están considerando, y cómo se comparan en rendimiento medido con Slint y con Flutter? Para cada opción: fuentes verificables con fecha, estado real en cada plataforma (en especial varios webviews nativos simultáneos en Android e iOS), madurez, licencia y riesgos.

Respuesta

Opciones no consideradas para la interfaz y para la gestión de varios webviews nativos (ChatCouncil, reescritura en Rust)

1. Pregunta entendida
   Identificar opciones viables hoy (noviembre de 2026) que no están en la evaluación actual (Flutter textura, Flutter ventanas nativas, Slint+wry, y descartes de Dioxus y Rune), que cumplan: un solo código, cinco plataformas (Windows, macOS, Linux, Android, iOS), núcleo en Rust, y rendimiento absoluto como criterio principal. Para cada una: estado real por plataforma, con foco en diez

webviews nativos simultáneos en Android e iOS, madurez, licencia, riesgos y comparación de rendimiento medido con Slint y Flutter.

2. Resumen ejecutivo (ordenado por relevancia)
   La limitación decisiva no es la biblioteca de UI, sino el móvil: wry y Tauri solo exponen varios webviews dentro de una ventana en escritorio; en Android e iOS su modelo es ventanas separadas (actividades y escenas). Ninguna opción de "un solo código" evaluada resuelve hoy diez webviews simultáneos en pantalla en un teléfono (docs de Tauri, docs.rs tauri Window::add_child, verificado).
   La única arquitectura con diez webviews nativos simultáneos de primera clase en Android e iOS hoy es la de caparazón nativo (Kotlin/Swift) con el núcleo en Rust (UniFFI): es la que evita el sobrecosto de +5 puntos de CPU que usted midió con texturas en Flutter, porque las vistas nativas no se copian (inferencia a partir de su propia medición; confianza moderada).
   Tauri 2.x es la opción no considerada más madura para orquestación de webviews, pero su UI es un webview (misma objeción que descartó a Dioxus) y en móvil no ofrece webviews hijos en una sola ventana (docs de Tauri, verificado). Su RAM medida en macOS con webviews reales: 309 a 348 MB pico, frente a 106 a 124 MB de Slint (benchmark de memoria de 15 frameworks, agosto de 2026, medido).
   egui es viable en las cinco plataformas como UI, pero es más pesado en RAM que Slint en mediciones independientes (135 MB frente a 30 MB en la misma aplicación con renderizador de software de Slint) y no tiene API documentada para incrustar webviews nativos (Sarrade, 23 de septiembre de 2026, medido).
   Makepad, Dioxus Native/Blitz, GPUI e iced quedan descalificados para el requisito de webviews: Makepad no tiene widget de WebView (su renderizador HTML propio no tiene WebView), Blitz no tiene motor de JavaScript (no puede ejecutar las páginas de los proveedores de IA), GPUI móvil es experimental y iced no tiene soporte móvil mantenido (makepad-html, README de Blitz, GPUI Kit móvil, issue 302 de iced, verificado).
   Qt vía cxx-qt cubre las cinco plataformas con webviews nativos en móvil, pero en Linux exige Qt WebEngine (Chromium integrado), pesa más que Slint en RAM medida (133 a 151 MB frente a 106 a 124 MB) y arrastra riesgo de licencia LGPL en iOS (documentación de Qt WebView, benchmark de memoria).
   Dato de plataforma favorable a su caso: en Android moderno todas las instancias de WebView de una app comparten un único proceso renderizador, lo que hace que diez instancias simultáneas sean técnicamente manejables con destrucción explícita; en iOS cada WKWebView genera su propio proceso WebContent, con costos reportados de 50 a 200 MB por instancia, lo que hace inviables diez instancias calientes en un iPhone (documentación oficial de Android, verificado; BestHub, reportado).

3. Método y límites
   Se realizaron once búsquedas desde cinco ángulos (marcos de UI en Rust, orquestación de webviews, documentos oficiales de plataforma, mediciones comparativas y hojas de ruta), y se

abrieron las fuentes decisivas (benchmark de memoria de Zenn, caso egui→Slint, documentación oficial de Tauri, Qt, Android y Slint). Límites: no se ejecutó ningún código sobre los candidatos (ninguna cifra de esta respuesta reemplaza su banco de pruebas en Windows); las páginas de documentación oficial rara vez llevan fecha de publicación, por lo que su vigencia es "consultado el 9 de noviembre de 2026"; no se verificó la licencia del repositorio ComposeNativeWebview ni el soporte iOS de su paquete; no se encontró ninguna medición pública de arranque o RAM para Makepad.

4. Panorama comparativo
   Opción
   Cinco plataformas
   Diez webviews simultáneos
   Madurez
   Licencia
   Tauri 2.x
   Sí, estable
   Escritorio: sí (inestable); móvil: no en una ventana
   Alta
   MIT/Apache-2.0
   egui
   Sí (móvil reciente)
   No (sin API de incrustación)
   Media-alta en escritorio
   MIT/Apache-2.0
   Makepad
   Sí
   No (sin WebView)
   1.0 de mayo de 2025, ecosistema joven
   MIT/Apache-2.0
   Dioxus Native/Blitz
   Escritorio + Android
   No (sin JavaScript)
   Beta
   MIT/Apache-2.0
   Qt vía cxx-qt
   Sí
   Sí en móvil; en Linux exige Chromium
   Alta (Qt); media (bindings)
   Bindings MIT/Apache; Qt LGPL/GPL/comercial
   Compose + wry (ComposeNativeWebview)

Sí
En teoría (por probar)
Proyecto joven
No verificada
Caparazón nativo + Rust (UniFFI)
Sí
Sí, de primera clase
Alta (por componente)
Por componente
GPUI / iced / Xilem
No (móvil)
No aplicable
Experimental / sin móvil
Varias

5. Detalle por opción
   Tauri 2.x (con wry) — madurez alta; no resuelve el móvil en una sola ventana
   Estado: soporte móvil estable desde la versión 2.0.0 (2 de octubre de 2024); la línea estable llega a 2.12 (26 de septiembre de 2026) y sigue recibiendo correcciones (reportado por análisis issoh, actualizado el 31 de octubre de 2026). Usa WebView2, WKWebView, WebKitGTK, WKWebView en iOS y WebView del sistema en Android (verificado en documentación de Tauri).
   Varios webviews: en escritorio, una ventana puede alojar cualquier cantidad de webviews hijos mediante Window::add_child, pero esa API está marcada "Available on desktop" y detrás de la funcionalidad inestable (verificado en docs.rs tauri). En móvil, tauri-runtime-wry 2.11.0 (30 de abril de 2026) añadió múltiples ventanas en Android (Activity Embedding, API 32+) e iOS (escenas, iOS 13+); la propia documentación advierte que en teléfonos la nueva ventana reemplaza a la actual y que la coexistencia real es una experiencia de tableta y iPad (notas de la versión, documentación Multi-Window on Mobile, verificado).
   Rendimiento medido: en el benchmark multi-módulo de agosto de 2026 (macOS, muestreo rsscap a 250 ms, método y repositorio públicos), Tauri ocupa 309 a 348 MB pico incluyendo sus procesos webview, frente a 106 a 124 MB de Slint y 176 a 199 MB de Flutter; en tamaño de paquete, una app vanilla Tauri 2.12 pesa 4,4 MB frente a 294 MB de Electron en macOS (benchmark Zenn, análisis issoh, medido).
   Riesgos: la UI de la aplicación es un webview más (la objeción que descartó a Dioxus); los webviews hijos son API inestable; en móvil no existe la cuadrícula de diez webviews en una sola vista. Tauri 3.0 (alfa desde el 13 de septiembre de 2026; alfa.5 el 14 de octubre de 2026) añade el runtime CEF, pero solo para escritorio, agranda el paquete al incluir Chromium, y está sin estabilizar ni sin ecosistema de complementos (documentación v3, análisis issoh, verificado). El runtime experimental Servo/Verso sigue sin publicarse en crates.io (reportado por análisis issoh).
   Caparazón nativo + núcleo Rust (UniFFI) — la única vía de primera clase para el requisito móvil

Estado: en Android, WebView es una vista nativa más y la documentación oficial describe explícitamente el ciclo de vida de múltiples instancias con destrucción explícita y recuento de instancias vivas; además, desde Android 8.0 todo el contenido web de la app corre en un único proceso renderizador aislado compartido, con la memoria en el espacio nativo y no en el montón Java (verificado en gestión de memoria de WebView y en la guía de memoria). En iOS, cada WKWebView corre en su propio proceso WebContent con un proceso de red compartido, y las guías de producción recomiendan agrupar y limitar instancias (reportado por BestHub, sin fuente primaria abierta en esta ronda).
Rendimiento: su propia medición en Windows ya muestra el patrón: ventanas nativas (Slint con wry, Flutter con ventanas nativas) cuestan casi cero CPU en el proceso propio frente a +5 puntos con texturas. La composición con vistas nativas en Android e iOS tiene la misma naturaleza de costo casi nulo; es una inferencia a partir de su medición y de la arquitectura, no una medición pública móvil (confianza moderada).
Riesgos: rompe el objetivo de "un solo código" para la capa de UI (Kotlin y Swift para el caparazón), aunque el núcleo, la automatización de webviews y el informe permanecen en un único binario Rust en las cinco plataformas. En iPhone, diez instancias calientes son inviables por jetsam: una prueba de estrés reportada con diez WebViews activos fue terminada en torno a un segundo, con 50 a 80 MB por página en blanco y 120 a 200 MB por página real (blog de Cross Mini App, 15 de octubre de 2026, reportado).
egui — UI viable en cinco plataformas, pero más pesada y sin webviews
Estado: eframe 0.30.0 (16 de diciembre de 2024) añadió soporte Android; 0.31.0 (4 de febrero de 2025) gestionó suspensión en Android e iOS; 0.37.0 (15 de octubre de 2026) es la versión actual; la mejora de teclado móvil de 0.36.0 aplica solo a eframe en web (verificado en CHANGELOG de eframe). El issue sobre móvil sigue abierto desde 2022, con usuarios confirmando Android funcional pero con integraciones de plataforma incompletas como el teclado nativo, e iOS sin documentar en el README (issue 2066, verificado).
Webviews: eframe renderiza por completo a través de su superficie wgpu o glow; no existe una API documentada de composición de vistas nativas para incrustar WKWebView o WebView, por lo que los diez webviews exigirían el mismo trabajo por plataforma que la opción de caparazón nativo (verificado por ausencia en la documentación oficial consultada; no es una lectura de código).
Rendimiento medido: en el benchmark de agosto de 2026, egui ocupa 148 a 157 MB pico (más unos 25 MB de fuente CJK que Slint no carga), frente a 106 a 124 MB de Slint; en un caso real documentado el 23 de septiembre de 2026, la misma aplicación pasó de ~135 MB con egui a ~30 MB con Slint y su renderizador de software, porque egui arrastra el contexto GPU y el atlas de texturas, y por modo inmediato repinta la UI completa en cada fotograma mientras Slint repinta solo lo sucio (benchmark Zenn, Sarrade, medido). Ambas mediciones son coherentes entre sí y con sus números de Windows.
Riesgo principal: en su caso de uso, egui redibuja su propia UI mientras diez páginas de proveedores redibujan; su prioridad de CPU en reposo favorece a Slint.

Opciones descalificadas para el requisito central (verificación incluida)
iced: la documentación de la versión 0.14.0 (7 de diciembre de 2025) declara soporte para Windows, macOS, Linux y web, sin móvil; el mantenedor declaró explícitamente que no le interesa soportar móviles (issue 302, abierto desde 2020), y el esfuerzo comunitario iced-mobile solo cubre Android y se declara "no afiliado" (docs.rs iced, issue 302, iced-mobile, verificado).
Makepad: versión 1.0 publicada el 15 de mayo de 2025, sobre macOS, Windows, Linux, iOS, Android y WASM, con licencia MIT/Apache y aplicaciones reales (Robrix, Moly); sin embargo, su renderizador HTML propio se describe explícitamente "sin WebView" y no incluye motor de JavaScript, la accesibilidad figura como trabajo pendiente y el ecosistema es joven (libro de Makepad, mayo de 2025, repositorio, makepad-html, verificado). No se encontró ninguna medición pública de RAM o arranque frente a Slint (no verificado).
Dioxus Native/Blitz: la objeción original a Dioxus (su UI es un webview) se corrige con Blitz, pero este es un motor de renderizado HTML/CSS en estado beta, deliberadamente sin motor de JavaScript, que hoy renderiza sitios sin JS y advierte de "muchos errores y funcionalidades faltantes"; las páginas reales de los proveedores de IA requieren JavaScript, sesiones y subida de archivos, por lo que Blitz no puede sustituir a los webviews, y como capa de UI no hay mediciones publicadas frente a Slint (README de Blitz, hoja de ruta: 0.1 jun-2025, 0.2 oct-2025, 0.3.0-beta.4 el 16 de octubre de 2026, lib.rs blitz-html, verificado). Dioxus 0.8 (plan del 24 de noviembre de 2025) apunta a APIs nativas y widgets SwiftUI/Kotlin (discusión 5024, verificado).
GPUI (Zed): el propio GPUI no está publicado en crates.io y el soporte móvil es experimental con IME, accesibilidad y ciclo de vida incompletos; la guía oficial del kit recomienda tratarlo como "no una garantía general de soporte móvil" (GPUI Kit Mobile, gpui-mobile en docs.rs, verificado). No viable para cinco plataformas hoy.
Qt vía cxx-qt — viable con costos
Estado: los bindings de KDAB están activos (0.8.0 el 18 de diciembre de 2025, 0.8.1 el 17 de febrero de 2026, línea 0.9-0.10 a mediados de 2026), con licencia MIT/Apache para los bindings y QtCore/QtGui cubiertos; no existen bindings Rust de QWidgets, así que la UI sería Qt Quick (repositorio cxx-qt, publicaciones, verificado). El módulo Qt WebView usa webviews nativos del sistema en Android, iOS y macOS, pero en Linux depende de Qt WebEngine (Chromium integrado) y en Windows puede usar WebEngine o WebView2 (verificado en documentación oficial de Qt WebView).
Rendimiento medido: Qt ocupa 133 a 151 MB pico en el benchmark de agosto de 2026, frente a 106 a 124 MB de Slint (misma fuente, mismo método), es decir, más pesado que Slint pero dentro del rango nativo y por debajo de Flutter (benchmark Zenn, medido).
Riesgos: licencia de Qt (LGPL/GPL/comercial) con la fricción habitual de LGPL en iOS (enlace dinámico); Qt WebEngine en Linux contradice la prioridad de tamaño; y no verifiqué en esta ronda los límites multi-instancia documentados de Qt WebView (la documentación oficial menciona una sección de limitaciones por plataforma que no se abrió).
Compose Multiplatform + webviews vía wry (ComposeNativeWebview)

Proyecto joven (182 estrellas, actividad hace dos o tres meses) que expone webviews nativos a Compose Multiplatform (Android, iOS, escritorio) a través de wry y UniFFI, sin empaquetar Chromium, con binarios más pequeños y arranque más rápido como objetivo declarado; incluye pruebas e2e visuales en Android y escritorio (repositorio, verificado como existencia y objetivo; la afirmación de rendimiento es del propio proyecto, no verificada). El costo heredado es la JVM: Compose Multiplatform en escritorio midió 283 a 565 MB pico en el benchmark de agosto de 2026, el peor grupo tras Electron y Fyne (benchmark Zenn, medido). Su licencia no fue verificada.

6. Hecho de plataforma clave para Android e iOS
   En Android, diez WebView simultáneos dentro de una actividad son técnicamente posibles y el sistema comparte un solo proceso renderizador entre todas las instancias de la app, pero la memoria es nativa, invisible al montón Java, y exige destrucción explícita (WebView.destroy) para evitar fugas y OOM (verificado en la documentación oficial). En iOS cada WKWebView genera su propio proceso WebContent, con costos reportados de 50 a 80 MB por instancia en blanco y 120 a 200 MB por página real, lo que obliga a agrupar y limitar instancias en iPhone y hace razonable el modelo de diez webviews solo en iPad, escritorio y tableta (reportado por BestHub y Cross Mini App; no se abrió fuente primaria de Apple en esta ronda, vacío declarado). Slint, su base actual, soporta oficialmente Android (Rust y C++) e iOS (solo Rust) (documentación de Slint, verificado), pero eso cubre la UI, no la incrustación multi-webview en móvil.
7. Conflicto, vacío y conclusión operativa
   Conflicto relevante: ninguna fuente pública contradice sus mediciones de Windows; el benchmark de agosto de 2026 y el caso de septiembre de 2026 las refuerzan (Slint por debajo de Flutter en RAM; frameworks con UI en webview, como Tauri, en otro orden de magnitud de RAM total con procesos incluidos).
   Vacío principal no cubierto: si su plan Slint+wry en Android e iOS permite crear varios webviews dentro de una misma ventana en móvil. wry documenta webviews hijos para macOS, Windows y Linux (X11), y Tauri reserva add_child para escritorio (README de wry, verificado); presumo que en móvil wry crea un webview por actividad y que diez simultáneos exigirían Kotlin/Swift propios, pero no lo verifiqué con una prueba ejecutable (no verificado; sería el primer chequeo a realizar).
   Conclusión operativa: si el requisito de diez webviews simultáneos en pantalla debe cumplirse también en teléfono, la única vía probada hoy es caparazón nativo + núcleo Rust, aceptando dos UI de caparazón; si basta tableta/iPad + escritorio, Slint+wry (o Tauri 2.x si se acepta la UI en webview) cubre el caso con el mejor perfil medido. Confianza: alta para las limitaciones documentadas de wry/Tauri en móvil; moderada para el costo casi nulo de la composición nativa móvil (inferido, no medido); desconocida para multi-webview de wry en móvil (sin prueba).
8. Próximos pasos sugeridos
   Prueba ejecutable de wry en Android e iOS: intentar dos webviews en una misma ventana/actividad; esto confirma o refuta el vacío principal con bajo costo.
   Prueba de jetsam en iPhone con 3, 5 y 10 WKWebView calientes con páginas reales de proveedores, para fijar el límite práctico de agrupación.

Si se evalúa Tauri 2.x como capa de orquestación, medir su RAM con diez webviews reales en la misma máquina de su banco de Windows, porque el benchmark citado mide apps de un solo webview.
Verificar la licencia de ComposeNativeWebview y su estado iOS antes de considerarlo como vía Compose.

Texto capturado tal cual de la interfaz del proveedor, sin corregir ni recortar.
Este documento es la respuesta a la pregunta, no la evaluacion que este proveedor hizo de las respuestas de los demas.